

LEGO® Set Inventory Management System



Overview

In this project, you will create an inventory management system for LEGO® sets using Python. The task is to process and manipulate this data, allowing the user to perform various actions, specified in Task Breakdown below. During your two-hour lab in Week 12 you will be asked to add functionality for menu options 4, 5, and 6. This will be open-book but no internet.

Instructions

- Use function to divide the tasks into smaller parts and promote re-use.
- Use a `main()` function as your driving function to avoid global variables, which are not allowed
- You may only use what we have learned in SOFT6017 and SOFT6018¹
- Use **Git Version Control** to log the progress of your application's development, and reflect the steady progress of your application.
- You have been provided with a module called **validation.py**. Familiarise yourself with the functions in this module, focusing on the sample uses in `if/main`. Use it when appropriate to validate inputs.

Submit in 2-hour lab Week 12

In Week 12, during your 2-hour lab, you will add functionality for options 4, 5 and 6 and then you will submit:

- **File 1**- the original code you worked on during the lab and at home
- **Images** of your Git log for development of File 1
- **File 2**- the same as File 1 but with your attempt at the in-lab requirements for menu options 4,5, and 6. No git is required for this part of the project.

¹ You may also use dictionaries because these were included as optional content.

Data Structure

A file contains data about LEGO® sets with their respective

- Set Number (unique 5-7 digit)
- Title
- number of pieces
- recommended retail price (RRP), with 0.00 for retired products
- stock status (1 for in stock, 0 for out of stock).

Sample data

```
10276,Colosseum,10279,0,1
10307,Eiffel Tower,10001,629.99,1
10253,Big Ben,4163,0.00,0
10325,Alpine Lodge,1517,99.99,1
10302,Optimus Prime,1508,179.99,1
21355,The Evolution of STEM,879,79.99,1
42177,Mercedes-Benz 500,2891,249.99,0
10300,Back to the Future Time Machine,1872,199.99,1
42140,App-Controlled Transformation Vehicle,772,0.00,1
10283,NASA Space Shuttle Discovery,2354,0.00,1
10362,French Café,1101,79.99,1
21348,Dungeons & Dragons: Red Dragon's Tale,3745,359.99,1
42121,Heavy Duty Excavator,569,0,1
76399,Hogwarts® Magical Trunk,603,0,0
42143,Ferrari Daytona SP3,3778,349.99,0
42156,Peugeot 9X8 24H Le Mans Hybrid Hypercar,1775,199.99,1
76439,Ollivanders® & Madam Malkin's Robes,744,89.99,1
60452,Donut Truck,196,19.99,1
60179,Ambulance Helicopter,190,0,1
42158,NASA Mars Rover Perseverance,1132,95.99,1
42159,Yamaha MT10SP,1478,229.99,0
76446,Knight Bus® Adventure,499,49.99,1
10345,Flower Arrangement,1161,109.99,1
71832,Thunderfang Dragon of Chaos,668,74.99,1
10218,Pet Shop,2032,0.00,0
10297,Boutique Hotel,3066,229.99,1
43268,Lilo and Stitch Beach House,834,89.99,1
```

Task Breakdown

1. Load the data from the file into data structures² using appropriate datatypes. Handle the file not existing and corrupt data by ending the application.

Write this as a function and call it in main().

2. Display a menu with 7 options where the 7th option is Quit. The menu repeats until the user chooses 7 - Quit.

1. Display Inventory
2. Add New LEGO® Set
3. Search by Code Prefix
4. Placeholder for Week 12 Lab
5. Placeholder for Week 12 Lab
6. Placeholder for Week 12 Lab
7. Quit

Enter your choice: 1

3. Complete the first 3 menu options and the Quit option as follows:

Menu Option 1.

Display the inventory in a neat table, marking retired sets with a special marker at the end of the line and including a column with  if in stock and  if not in stock³.

Write this as a function that receives the list(s) as parameters.

Call this function in Option 1 of the menu.

A sample of the output might be:

Lego Sets			

10276 Colosseum	10,279		0.00#
10307 Eiffel Tower	10,001		629.99
10253 Big Ben	4,163		0.00#
10325 Alpine Lodge	1,517		99.99
10302 Optimus Prime	1,508		179.99
21355 The Evolution of STEM	879		79.99
42177 Mercedes-Benz 500	2,891		249.99

Menu Option 2.

Add a new LEGO® set to the list(s).

Create this as a function that receives the list(s) as parameters.

² Data structures here might be parallel lists, or lists of objects (use classes) or lists of dictionaries or dictionary of dictionaries – this is your choice.

³ Or use your own preferred symbols.

- Inside the function ask the user for the data about the new set. Validate all inputs using functions from validation.py and/or your own functions.
 - Ensure that the LEGO® set number provided does not already exist. Handle the situation if it does exist by print an error message and exiting the function without appending the data.
 - If the LEGO® set number is unique, append the data to the list(s)
 - Print a success message when done.
- Call this function in Option 2 of the menu.

Menu Option 3.

Create a function that receives the list(s) and a string (pattern) and **returns** the names and set numbers of those LEGO® sets whose set numbers start with the pattern.

Usage? A user may be searching for set numbers that start with "76" because these are the Harry Potters sets. Or perhaps those that start with "10" because these are the Icons® sets.

In Option 3 of the menu, ask the user for the desired starting pattern and call the function passing it the list(s) and the pattern. Display the returned data or, if no sets are found, print an appropriate message.

```

1. Display Inventory
2. Add New LEGO® Set
3. Search by Code Prefix
4. Placeholder for Week 12 Lab
5. Placeholder for Week 12 Lab
6. Placeholder for Week 12 Lab
7. Quit
>>> 3
Which code are you looking for? 99
No sets whose code starts with 99

1. Display Inventory
2. Add New LEGO® Set
3. Search by Code Prefix
4. Placeholder for Week 12 Lab
5. Placeholder for Week 12 Lab
6. Placeholder for Week 12 Lab
7. Quit
>>> 3
Which code are you looking for? 76

Sets whose code starts with 76:
-----
76399  Hogwarts™ Magical Trunk
76439  Ollivanders™ & Madam Malkin's Robes
76446  Knight Bus™ Adventure

```

Menu Option 4. Placeholder for in-lab project work

Menu Option 5. Placeholder for in-lab project work

Menu Option 6. Placeholder for in-lab project work

Menu Option 7. Quit - When the user quits, all data should be written from the lists back to the file to persist the edits made.